

LOW-LEVEL DESIGN

User Posts & Albums Viewer

Client-side web application using HTML, CSS, JavaScript and jQuery

Document Version	0.2
Status	Revised Draft - review feedback incorporated
Prepared By	Atul Sharma
Technology	HTML5, CSS3, JavaScript (ES6+), jQuery 4.0.0
Data Source	JSONPlaceholder REST API

Revision Summary

Review update: The LLD has been updated to make the complete API URLs, representative expected response structures, and name-display handling explicit. The existing independent/concurrent Posts and Albums request design remains unchanged.

1. Purpose

This Low-Level Design translates the confirmed requirements into an implementation-level design for the User Posts & Albums Viewer. It defines the client-side project structure, functions, DOM structure, API interactions, state handling, event handling, loading/error behavior, and rendering responsibilities.

The design remains intentionally lightweight because the confirmed scope contains no custom backend, database, authentication, or persistent storage. JSONPlaceholder is the only external data source.

2. Requirements Baseline

ID	Confirmed requirement	LLD response
FR-01	Retrieve users from JSONPlaceholder	getUsers() + loadUsers()
FR-02	Display first name + last name	renderUsers() uses API name display value; see Section 10.1
FR-03	Allow user selection	Selectable user controls + delegated click handling
FR-04	Retrieve selected user posts	getUserPosts(userId)
FR-05	Retrieve selected user albums	getUserAlbums(userId)
FR-06	Posts and Albums are separate	activeView + renderPosts() /

ID	Confirmed requirement	LLD response
		renderAlbums(); one shared content area
FR-07	Loading feedback	Separate users/details loading elements and state flags
FR-08	User-facing API error handling	Separate users/details error elements and failure handlers
FR-09	Update without full page refresh	jQuery DOM updates after requests complete

3. Scope and Design Boundaries

3.1 In scope

- Initial retrieval and rendering of the user list.
- Display of each user first and last name as the requested display name.
- User selection.
- Retrieval of both posts and albums for the selected user.
- Separate Posts and Albums controls/views.
- Loading, error, and empty states.
- Client-side DOM rendering.
- Basic responsive/presentable CSS.

3.2 Out of scope

- Custom backend or server-side application code.
- Custom database or persistence.
- Authentication and authorization.
- User registration/login.
- CRUD operations against an application-owned data store.
- Advanced UI/UX beyond the confirmed feature structure.

4. Proposed Project Structure

```

user-posts-albums-viewer/
|
+- index.html
+- css/
| +- style.css
+- js/
| +- api.js
| +- ui.js
| +- app.js
+- docs/
| +- RUD_User_Posts_Albums_v1.1.pdf
| +- SRS_User_Posts_Albums_v0.1.pdf
| +- API_Spec_User_Posts_Albums_v0.1.pdf
| +- HLD_User_Posts_Albums_v0.2.pdf
| +- LLD_User_Posts_Albums_v0.2.pdf
+- README.md

```

File	Responsibility
index.html	Static document structure and stable UI containers
css/style.css	Layout, responsive behavior, selected state, loading/error/empty styling
js/api.js	JSONPlaceholder request functions only
js/ui.js	DOM rendering and UI state updates only
js/app.js	State, event handling, orchestration, and request coordination
README.md	Setup, usage, and project overview

Design rule: The separation is deliberately small; no framework or additional state-management library is introduced.

5. Runtime Flow

1. On document ready, `loadUsers()` starts the initial request.
2. The client requests `GET https://jsonplaceholder.typicode.com/users`.
3. Returned users are stored in memory and rendered using the `name display` value.
4. The user selects a user control.
5. The selected user is stored and previous user-specific data is cleared.
6. Two independent requests start for the selected user: Posts and Albums. They are separate API calls and may execute concurrently.
7. When both responses are available, the client stores both datasets and enables the separate view controls.
8. The active view is rendered into a single content area.

6. Client-Side State Model

```
const state = {  
  users: [],  
  selectedUser: null,  
  posts: [],  
  albums: [],  
  activeView: "posts",  
  usersLoading: false,  
  detailsLoading: false,  
  usersError: null,  
  detailsError: null,  
  detailRequestId: 0  
};
```

State field	Type	Responsibility
users	Array	Retrieved user records
selectedUser	Object / null	Current selected user
posts	Array	Posts for the selected user
albums	Array	Albums for the selected user
activeView	String	posts or albums
usersLoading	Boolean	Initial /users request state
detailsLoading	Boolean	Selected-user request state
usersError	String / null	Initial users error state
detailsError	String / null	Selected-user error state
detailRequestId	Number	Prevents stale responses from a previous selection from updating the current UI

activeView defaults to posts to match the sample UI state; the user can switch to albums after the selected user detail data is available.

7. HTML / DOM Structure

```

<body>
  <header>
    <h1>User Posts & Albums Viewer</h1>
  </header>
  <main>
    <section id="users-panel">
      <h2>Users</h2>
      <div id="users-loading" hidden></div>
      <div id="users-error" hidden></div>
      <div id="users-list"></div>
    </section>
    <section id="details-panel">
      <h2 id="selected-user"></h2>
      <div id="content-controls">
        <button id="posts-btn" type="button">Posts</button>
        <button id="albums-btn" type="button">Albums</button>
      </div>
      <div id="loading-message" hidden></div>
      <div id="error-message" hidden></div>
      <div id="content-area"></div>
    </section>
  </main>
</body>

```

Element	Responsibility
#users-list	Render selectable users
#users-loading	Show initial user-list loading state

Element	Responsibility
#users-error	Show initial user-list error state
#selected-user	Show selected user display name
#posts-btn	Activate Posts view
#albums-btn	Activate Albums view
#loading-message	Show selected-user loading state
#error-message	Show selected-user error state
#content-area	Display either Posts or Albums, never both together

8. API Layer (api.js)

api.js contains transport logic only. It must not modify the DOM.

8.1 Base URL

```
const API_BASE_URL = "https://jsonplaceholder.typicode.com";
```

8.2 Retrieve users

Complete request URL: <https://jsonplaceholder.typicode.com/users>

```
function getUsers() {
  return $.ajax({
    url: `${API_BASE_URL}/users`,
    method: "GET",
    dataType: "json"
  });
}
```

8.3 Retrieve posts

Complete request URL pattern: <https://jsonplaceholder.typicode.com/users/{userId}/posts>

Example for userId = 1: <https://jsonplaceholder.typicode.com/users/1/posts>

```
function getUserPosts(userId) {
  return $.ajax({
    url: `${API_BASE_URL}/users/${userId}/posts`,
    method: "GET",
    dataType: "json"
  });
}
```

8.4 Retrieve albums

Complete request URL pattern: <https://jsonplaceholder.typicode.com/users/{userId}/albums>

Example for userId = 1: <https://jsonplaceholder.typicode.com/users/1/albums>

```
function getUserAlbums(userId) {
  return $.ajax({
    url: `${API_BASE_URL}/users/${userId}/albums`,
    method: "GET",
    dataType: "json"
  });
}
```

API-layer rule: The API layer returns the result of the jQuery AJAX operation. UI rendering, loading messages, and error presentation are handled by app.js and ui.js.

8.5 Expected API Results

The following are representative response structures showing the fields consumed by this UI. The API may return additional fields; the renderer uses only the fields listed below.

Users - GET /users

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "email": "Sincere@april.biz"
  }
]
```

Posts - GET /users/{userId}/posts

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "...",
    "body": "..."
  }
]
```

Albums - GET /users/{userId}/albums

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "..."
  }
]
```

8.6 API Result Handling Rules

- A successful response is processed as JSON.
- A non-success HTTP response or failed network request is treated as an API failure.

- The UI never assumes that Posts and Albums come from the same response; they are separate response payloads.

9. Application Logic (app.js)

app.js owns state, event handling, request orchestration, and selection logic.

9.1 Initialization

```
$(function () {
  loadUsers();
  bindEvents();
});
```

9.2 Initial user retrieval

The initial request has its own loading and error state so the users area can communicate the request status independently of selected-user details.

```
function loadUsers() {
  state.usersLoading = true;
  state.usersError = null;
  showUsersLoading();
  clearUsersError();
  return getUsers()
    .done(function (users) {
      state.users = users;
      renderUsers(state.users);
    })
    .fail(function () {
      state.usersError = "Failed to fetch users.";
      showUsersError(state.usersError);
    })
    .always(function () {
      state.usersLoading = false;
      hideUsersLoading();
    });
}
```

9.3 User selection

```
function handleUserSelection(user) {
  if (!user || !user.id) {
    showError("Unable to identify the selected user.");
    return;
  }

  state.selectedUser = user;
  state.posts = [];
  state.albums = [];
  state.activeView = "posts";
  state.detailsError = null;
  state.detailRequestId += 1;

  $("#selected-user").text(user.name);
  clearContentArea();
  clearError();
  loadUserDetails(user.id, state.detailRequestId);
}
```

9.4 Loading posts and albums

Request behavior: Posts and Albums are two different API requests. They are initiated independently and concurrently because neither request depends on the other response.

```
function loadUserDetails(userId, requestId) {
  state.detailsLoading = true;
  showLoading();
  clearError();
  setContentControlsEnabled(false);

  return $.when(
    getUserPosts(userId),
    getUserAlbums(userId)
  )
  .done(function (posts, albums) {
    if (requestId !== state.detailRequestId) { return; }

    state.posts = posts[0];
    state.albums = albums[0];
    state.detailsError = null;
    setContentControlsEnabled(true);
    renderActiveView();
  })
  .fail(function () {
    if (requestId !== state.detailRequestId) { return; }

    state.detailsError = "Failed to load user details.";
    clearContentArea();
    showError(state.detailsError);
  })
  .always(function () {
    if (requestId === state.detailRequestId) {
      state.detailsLoading = false;
      hideLoading();
    }
  });
}
```

The request ID prevents an earlier selection from overwriting the UI after the user has already selected another user.

9.5 View switching

```
function handleViewChange(view) {
  if (state.detailsLoading || state.detailsError) {
    return;
  }
  state.activeView = view;
  clearError();
  renderActiveView();
}

function renderActiveView() {
  if (state.activeView === "posts") {
    renderPosts(state.posts);
  } else {
    renderAlbums(state.albums);
  }
}
```

10. UI Rendering Layer (ui.js)

ui.js performs DOM updates and contains no API transport logic.

10.1 Render users and name handling

Name handling: JSONPlaceholder exposes a single name field for each user. The value is a full display name such as "Leanne Graham"; the current assignment uses this full value as the required first-name + last-name presentation. The application does not split the string into separate firstName and lastName fields because no separate fields or parsing rule are defined in the requirements.

```
function renderUsers(users) {
  const $list = $("#users-list");
  $list.empty();
  $.each(users, function (_, user) {
    const $button = $("", {
      type: "button",
      class: "user-item",
      text: user.name,
      "data-user-id": user.id
    });
    $list.append($button);
  });
}
```

10.2 Render Posts

```
function renderPosts(posts) {
  const $content = $("#content-area");
  $content.empty();
  const $heading = $("

### ").text(`Posts: ${posts.length}`); $content.append($heading); if (posts.length === 0) { $content.append($("<p>").text("No posts available for this user.")); return; } const $list = $("<ul>"); $.each(posts, function (_, post) { const $item = $("<li>"); $("<strong>").text(post.title).appendTo($item); $("<p>").text(post.body).appendTo($item); $item.appendTo($list); }); $content.append($list); }


```

10.3 Render Albums

```
function renderAlbums(albums) {
  const $content = $("#content-area");
  $content.empty();
  const $heading = $("

### ").text(`Albums: ${albums.length}`); $content.append($heading); if (albums.length === 0) { $content.append($("<p>").text("No albums available for this user.")); } }


```

```

    return;
}

const $list = $("<ul>");
$.each(albums, function (_, album) {
    $("<li>").text(album.title).appendTo($list);
});
$content.append($list);
}

```

10.4 Loading and error feedback

```

function showUsersLoading() {
    $("#users-loading").text("Loading users...").prop("hidden", false);
}

function hideUsersLoading() {
    $("#users-loading").prop("hidden", true);
}

function showLoading() {
    $("#loading-message").text("Loading posts and albums...").prop("hidden", false);
}

function hideLoading() {
    $("#loading-message").prop("hidden", true);
}

function showUsersError(message) {
    $("#users-error").text(message).prop("hidden", false);
}

function showError(message) {
    $("#error-message").text(message).prop("hidden", false);
}

```

```

function clearUsersError() {
    $("#users-error").prop("hidden", true).empty();
}

function clearError() {
    $("#error-message").prop("hidden", true).empty();
}

function clearContentArea() {
    $("#content-area").empty();
}

function setContentControlsEnabled(enabled) {
    $("#posts-btn, #albums-btn").prop("disabled", !enabled);
}

```

11. Event Handling

Because the user controls are created dynamically, delegated event handling is used on the stable `#users-list` container.

```

function bindEvents() {

```

```

$("#users-list").on("click", ".user-item", function () {
  const userId = Number($(this).data("user-id"));
  const user = state.users.find(function (item) {
    return item.id === userId;
  });
  handleUserSelection(user);
});

$("#posts-btn").on("click", function () {
  handleViewChange("posts");
});

$("#albums-btn").on("click", function () {
  handleViewChange("albums");
});
}

```

Event flow: User clicks user -> delegated click handler -> obtain user ID -> find user in state -> handleUserSelection(user) -> start independent Posts + Albums requests.

12. Detailed User-Selection Sequence

9. Selected user is updated immediately.
10. Old content is cleared.
11. A new detailRequestId value is generated.
12. Posts and Albums requests start independently.
13. Responses are accepted only if the request ID matches the current selection.
14. Once both responses succeed, both datasets are retained and the active view is rendered.

13. Posts / Albums View Logic

[Posts] [Albums]

Shared content area:

```

activeView = "posts" -> renderPosts(state.posts)
activeView = "albums" -> renderAlbums(state.albums)

```

The content area is shared, but only one view is rendered at any time. Switching views after the two datasets have been loaded does not create another API request; it only changes the rendered content.

14. Loading, Error, and Empty-State Handling

14.1 Initial users loading

```

Application start -> Loading users... -> GET /users -> success -> render user list
                  -> failure -> show users error

```

14.2 Selected-user loading

```

User selected -> clear old detail content -> Loading posts and albums...
              -> GET posts + GET albums (independent)
              -> both succeed -> enable controls + render active view

```

-> failure -> hide loading + show error

14.3 Empty data

A successful response with an empty array should not produce a blank content area.

- No posts available for this user.
- No albums available for this user.

14.4 Stale-response protection

When the user selects another user before the previous requests finish, `detailRequestId` ensures that an older result is ignored. This prevents the previously selected users posts or albums from being rendered under the newly selected users name.

15. Data Shapes and Expected UI Fields

User

```
{
  "id": 1,
  "name": "Leanne Graham"
}
```

Used fields: id, name

Post

```
{
  "userId": 1,
  "id": 1,
  "title": "...",
  "body": "..."
}
```

Used fields: id, title, body

Album

```
{
  "userId": 1,
  "id": 1,
  "title": "..."
}
```

Used fields: id, title

Response mapping: The expected result is an array for each endpoint. Each response is stored separately: users -> state.users, posts -> state.posts, albums -> state.albums.

16. Request and Concurrency Behavior

Posts and Albums are independent resources. The client therefore starts both requests without waiting for one request to complete before starting the other. The complete request patterns are:

- GET <https://jsonplaceholder.typicode.com/users/{userId}/posts>
- GET <https://jsonplaceholder.typicode.com/users/{userId}/albums>

```
return $.when(
  getUserPosts(userId),
  getUserAlbums(userId)
);
```

The currently selected users data is kept only in memory. No persistent cache is required.

17. Security Considerations

- API-provided text is treated as untrusted input for rendering.
- jQuery .text() is preferred for API-provided values rather than injecting raw HTML strings.
- No API credentials, tokens, or authentication secrets are required for this assignment.
- No authentication or authorization layer is included because it is outside the confirmed scope.

18. Maintainability and Design Rules

- Keep API communication in api.js.
- Keep DOM rendering in ui.js.
- Keep state and workflow orchestration in app.js.
- Use stable IDs/classes for DOM targeting.
- Clear or replace current content before rendering a new dataset.
- Use the selected user ID to construct user-specific endpoint URLs.
- Do not combine Posts and Albums into one view.
- Keep the two API calls independent; coordinate them in application logic rather than in the API layer.
- Avoid unnecessary abstraction because the assignment has a small scope.

19. Requirement Traceability

RUD/SRS requirement	LLD implementation
Retrieve users	getUsers() + loadUsers()
Display first + last name	renderUsers() using user.name; Section 10.1 explains the name mapping
Select user	Delegated click handler + handleUserSelection()
Retrieve posts	getUserPosts(userId) -> complete URL pattern in Section 8.3
Retrieve albums	getUserAlbums(userId) -> complete URL pattern in Section 8.4
Separate Posts/Albums	activeView, renderPosts(), renderAlbums()

RUD/SRS requirement	LLD implementation
Loading feedback	showUsersLoading(), showLoading() and corresponding hide functions
Error handling	Separate users/details error handlers
Dynamic update	jQuery DOM updates without full page reload

20. Implementation Checklist

- ☐ jQuery 4.0.0 is loaded.
- ☐ GET <https://jsonplaceholder.typicode.com/users> works on document ready.
- ☐ Users display the API name value as first-name + last-name display text.
- ☐ User controls are selectable.
- ☐ Selected user name is shown.
- ☐ Posts request uses <https://jsonplaceholder.typicode.com/users/{userId}/posts>.
- ☐ Albums request uses <https://jsonplaceholder.typicode.com/users/{userId}/albums>.
- ☐ Posts and Albums requests are independent and may run concurrently.
- ☐ Expected response arrays map to separate state.posts and state.albums collections.
- ☐ Posts and Albums are separate views.
- ☐ Switching views does not combine the two datasets.
- ☐ Loading feedback is visible during retrieval.
- ☐ Initial users errors are shown in the users area.
- ☐ Selected-user errors are shown in the details area.
- ☐ Empty arrays produce meaningful empty states.
- ☐ Selecting another user replaces previous detail data.
- ☐ Stale responses from older selections are ignored.
- ☐ No full-page refresh is required for user selection.
- ☐ API-provided text is rendered safely.
- ☐ No console errors remain during normal use.

21. Design Decisions and HLD/LLD Boundary

- jQuery is used for AJAX, event handling, and DOM manipulation.
- API functions are separated from UI rendering functions.
- Posts and Albums are requested as two independent calls after user selection, with concurrent initiation using \$.when().
- Both datasets are stored in memory for the current selection so view switching does not require another API request.
- A request ID prevents stale responses from changing the UI after a new user is selected.
- Posts and Albums use separate views in a shared content area.
- The API name field is used as the full first-name + last-name display value; no unsupported name-parsing rule is introduced.

22. Revision Notes - v0.2

Area	Change made
Complete API URLs	Added the base URL plus complete request URL patterns and a <code>userId=1</code> example for Posts and Albums.
Expected results	Added representative Users, Posts, and Albums JSON response structures and the UI fields consumed from each.
Name handling	Explicitly documented that <code>JSONPlaceholder</code> returns a single name field and that <code>user.name</code> is used for the required full-name display.
Independent requests	Retained and clarified the existing concurrent Posts and Albums request behavior; no combined API operation is introduced.

23. Document Status

LLD v0.2 - REVISED DRAFT. This revision incorporates the review feedback while preserving the confirmed client-side architecture and the existing independent Posts and Albums request design. It adds explicit endpoint URLs, representative response results, and the name-display handling required for implementation clarity.